

Reporte de Práctica: Cola de Impresión

Estructuras de Datos – Cola Estática y Cola Dinámica

Sesión 1 – Cola Estática

Implementación de la Cola de Impresión

En la primera sesión se implementó una simulación de cola de impresión utilizando una estructura de datos estática basada en arreglos. La cola se representó mediante la estructura `ColaLineal`, la cual almacena un arreglo de tareas con tamaño máximo definido por la constante `TAM_MAX`.

```
typedef struct {
    Tarea datos[TAM_MAX];
    int size;
} ColaLineal;
```

Esta estructura mantiene todas las tareas dentro de un arreglo y utiliza la variable `size` para indicar cuántos elementos hay actualmente en la cola. El uso de memoria estática implica que el tamaño máximo de la cola está definido en tiempo de compilación y no puede cambiar durante la ejecución del programa.

Tipos de Datos Utilizados

El programa utiliza estructuras (`struct`) y enumeraciones (`enum`) para representar los elementos del sistema de impresión. La estructura principal es `Tarea`, que agrupa todos los datos necesarios para representar un trabajo de impresión:

```
typedef struct {
    int id;
    char usuario[32];
    char documento[32];
    int total_pgs;
    int restante_pgs;
    int copias;
    Prioridad prioridad;
    Estado estado;
} Tarea;
```

El uso de `struct` permite agrupar diferentes tipos de datos en una sola entidad lógica, lo cual facilita la manipulación de los trabajos de impresión. Además, se utilizan enumeraciones para representar valores específicos del sistema:

```
typedef enum {  
    NORMAL = 0,  
    URGENTE = 1  
} Prioridad;
```

```
typedef enum {  
    EN_COLA = 0,  
    IMPRIMIENDO = 1,  
    COMPLETADO = 2,  
    CANCELADO = 3  
} Estado;
```

El uso de `enum` permite evitar el uso de números mágicos dentro del código, haciendo que el programa sea más legible y fácil de mantener.

Subprogramas y Contratos de Funciones

El programa está organizado mediante varias funciones que implementan las operaciones de la cola. Cada función tiene un propósito específico y sigue un contrato claro sobre cómo interactúa con la estructura de datos.

Por ejemplo, la función `q_enqueue` permite insertar una nueva tarea en la cola:

```
int q_enqueue(ColaLineal *cola, Tarea t)
```

Esta función recibe un puntero a la cola, ya que necesita modificar su contenido agregando un nuevo elemento. Si la cola está llena, la función retorna `0`; en caso contrario, inserta la tarea y retorna `1`.

Por otro lado, funciones como `q_is_empty` utilizan el modificador `const`:

```
int q_is_empty(const ColaLineal *cola)
```

El uso de `const` indica que la función solo leerá los datos de la estructura sin modificarlos, lo que ayuda a evitar errores y respeta el principio de menor privilegio en el diseño de funciones.

Operaciones de la Cola

La cola implementa las operaciones fundamentales de una estructura FIFO (*First In, First Out*).

Inicialización

La función `q_init` inicializa la cola estableciendo su tamaño en cero, indicando que inicialmente no hay tareas en espera:

```
void q_init(ColaLineal *cola)
{
    cola->size = 0;
}
```

Inserción de Tareas

La función `q_enqueue` agrega una nueva tarea al final de la cola:

```
cola->datos[cola->size] = t;
cola->size++;
```

El nuevo elemento se inserta en la posición indicada por `size`, y posteriormente se incrementa el contador de elementos.

Consulta del Siguiente Elemento

La función `q_peek` permite observar la siguiente tarea sin eliminarla de la cola:

```
*t = cola->datos[0];
```

Esto permite al sistema visualizar qué trabajo será procesado a continuación.

Eliminación de Tareas

La función `q_dequeue` elimina el primer elemento de la cola. Después de extraer la primera tarea, el programa desplaza todos los elementos del arreglo una posición hacia adelante:

```
for(int i = 1; i < cola->size; i++)
{
    cola->datos[i-1] = cola->datos[i];
}
```

Finalmente, el tamaño de la cola se reduce en una unidad.

Complejidad de las Operaciones

Operación	Complejidad
-----------	-------------

Operación	Complejidad
Enqueue (insertar)	O(1)
Peek (consultar)	O(1)
Dequeue (eliminar)	O(n)

La operación **dequeue** tiene complejidad O(n) debido a que requiere desplazar todos los elementos del arreglo para mantener el orden de la cola. Este comportamiento representa una de las principales limitaciones de las implementaciones estáticas basadas en arreglos.

Sesión 2 – Cola Dinámica

Estructura de Datos Dinámica

En la segunda sesión se implementó la cola de impresión utilizando memoria dinámica mediante listas enlazadas. A diferencia de la versión estática, cada elemento se almacena en un nodo asignado dinámicamente en memoria.

La estructura del nodo se define de la siguiente manera:

```
typedef struct Node_t{
    Tarea job;
    struct Node_t* next;
} Node_t;
```

Cada nodo contiene una estructura **Tarea** con la información del trabajo de impresión y un puntero **next** que apunta al siguiente nodo. La cola dinámica se representa mediante:

```
typedef struct{
    Node_t* head;
    Node_t* tail;
    int size;
} QueueDynamic_t;
```

El puntero **head** representa el frente de la cola, mientras que **tail** apunta al último elemento. Esta implementación permite que la cola crezca dinámicamente dependiendo de la cantidad de trabajos agregados.

Gestión de Memoria (Stack vs Heap)

En el programa se utilizan dos tipos de memoria: *stack* y *heap*.

La estructura principal de la cola se almacena en el *stack*, ya que es una variable local dentro de **main**:

```
QueueDynamic_t cola;
```

Cada nodo se crea dinámicamente con `malloc` y se almacena en el *heap*:

```
Node_t* nuevo = (Node_t*)malloc(sizeof(Node_t));
```

Es importante liberar esta memoria cuando ya no se necesita. Para ello se implementó `qd_destroy`:

```
void qd_destroy(QueueDynamic_t* q)
{
    Node_t* actual = q->head;

    while(actual != NULL)
    {
        Node_t* temp = actual;
        actual = actual->next;
        free(temp);
    }

    q->head = NULL;
    q->tail = NULL;
    q->size = 0;
}
```

Sin esta función existiría una fuga de memoria (*memory leak*), ya que los nodos permanecerían ocupando memoria incluso después de terminar el programa.

Alcance y Duración de Variables

La variable `id` dentro de la función `elegir` posee duración automática; existe únicamente durante la ejecución de la función y se destruye al terminar:

```
int id = 1;
```

Por otro lado, los nodos creados con `malloc` tienen duración dinámica: permanecen en memoria hasta que el programador los libera manualmente con `free`, lo que permite que los trabajos de impresión continúen existiendo en la cola incluso después de que la función que los creó haya finalizado.

Subprogramas y Contratos de Función

Las funciones que modifican la estructura reciben un puntero a la cola:

```
int qd_enqueue(QueueDynamic_t* q, Tarea job)
```

Las funciones que solo consultan utilizan puntero constante:

```
int qd_is_empty(const QueueDynamic_t* q)
```

El uso de `const` garantiza que la función no modificará los datos de la cola, respetando el principio de menor privilegio.

Operaciones de la Cola Dinámica

Inserción (Enqueue)

La función `qd_enqueue` crea un nuevo nodo y lo agrega al final de la cola manteniendo el orden FIFO:

```
q->tail->next = nuevo;  
q->tail = nuevo;
```

Consulta (Peek)

La función `qd_peek` permite observar el primer elemento sin eliminarlo:

```
*out = q->head->job;
```

Eliminación (Dequeue)

La función `qd_dequeue` elimina el primer nodo y libera su memoria:

```
Node_t* temp = q->head;  
q->head = temp->next;  
free(temp);
```

Si la cola queda vacía, también se actualiza el puntero `tail`.

Complejidad de las Operaciones

Operación	Complejidad
Enqueue	O(1)

Operación	Complejidad
Peek	O(1)
Dequeue	O(1)

A diferencia de la implementación estática, en la cola dinámica solo se actualiza el puntero `head` al eliminar, reduciendo el costo de $O(n)$ a $O(1)$.

Comparativa Final

Característica	Cola Estática	Cola Dinámica
Memoria utilizada	Stack (arreglo fijo)	Heap (nodos dinámicos)
Tamaño máximo	Fijo (TAM_MAX)	Ilimitado
Complejidad Dequeue	$O(n)$	$O(1)$
Gestión de memoria	Automática	Manual (malloc/free)
Riesgo de memory leak	No	Sí (si no se usa free)

Sesión 3 – Simulación del Proceso de Impresión

Simulación de la Cola de Impresión

En la tercera sesión se implementó una simulación completa del proceso de impresión utilizando la cola dinámica desarrollada en la sesión anterior. El objetivo fue representar cómo una impresora procesa cada trabajo página por página hasta completar todos los documentos en la cola.

La simulación se implementó mediante la función `simular_impresion`, la cual procesa todos los trabajos existentes en la cola hasta que esta queda vacía:

```
void simular_impresion(QueueDynamic_t *cola)
```

El algoritmo verifica si la cola está vacía y, en caso de haber trabajos pendientes, entra en un ciclo `while` que continúa hasta que todos hayan sido procesados:

```
while(!qd_is_empty(cola))
```

En cada iteración se extrae el siguiente trabajo con `qd_dequeue` y su estado se actualiza a `IMPRIMIENDO`:

```
qd_dequeue(cola, &t);
t.estado = IMPRIMIENDO;
```

Simulación del Progreso por Página

Para simular el proceso real de impresión, el programa utiliza un ciclo que reduce el número de páginas restantes hasta llegar a cero:

```
while(t.restante_pgs > 0)
```

En cada iteración se muestra en pantalla la página actual que está siendo impresa:

```
printf("Pagina %d de %d\n",
      t.total_pgs - t.restante_pgs + 1,
      t.total_pgs);
```

Luego el contador de páginas restantes se reduce en una unidad y se utiliza `sleep(1)` para simular el tiempo real de impresión:

```
t.restante_pgs--;
sleep(1);
```

Cuando las páginas restantes llegan a cero, el estado del trabajo cambia a `COMPLETADO`:

```
t.estado = COMPLETADO;
```

Transición de Estados de los Trabajos

Durante la simulación, cada trabajo pasa por diferentes estados que representan su progreso dentro del sistema:

Estado	Descripción
EN_COLA	El trabajo está esperando ser procesado
IMPRIMIENDO	El documento está siendo impreso
COMPLETADO	El documento terminó de imprimirse
CANCELADO	El trabajo fue eliminado antes de imprimirse

El flujo normal de un trabajo sigue la siguiente transición:

```
EN_COLA → IMPRIMIENDO → COMPLETADO
```

En caso de cancelación:

```
EN_COLA → CANCELADO
```

Mejora 1: Manejo de Prioridad Urgente

Una de las mejoras implementadas fue el manejo de prioridades. Los trabajos pueden clasificarse como **NORMAL** o **URGENTE**. Si un trabajo tiene prioridad urgente, se inserta al inicio de la cola para ser procesado antes que los demás:

```
else if(job.prioridad == URGENTE) {
    nuevo->next = q->head;
    q->head = nuevo;
}
```

Gracias a esta modificación, los trabajos urgentes se procesan inmediatamente después del trabajo actual, sin esperar a que todos los trabajos normales terminen.

Mejora 2: Cancelación de Trabajos

Otra mejora implementada fue la capacidad de cancelar trabajos mediante su identificador (`id`). La función `qd_cancel` busca el trabajo dentro de la lista enlazada y lo elimina si existe:

```
int qd_cancel(QueueDynamic_t* q, int id_buscar)
```

El algoritmo recorre la lista con dos punteros: `actual` (nodo actual) y `anterior` (nodo previo). Al encontrar el nodo con el ID correspondiente, se reconectan los punteros para eliminarlo de la lista:

```
anterior->next = actual->next;
```

Finalmente se libera la memoria del nodo con `free`, evitando fugas de memoria y asegurando que el trabajo cancelado no sea procesado durante la simulación:

```
free(actual);
```

Decisiones de Implementación

Durante el desarrollo del sistema se tomaron varias decisiones para mejorar la robustez y funcionamiento del programa.

Validaciones de Cola Vacía

Antes de realizar operaciones como `dequeue`, `peek` o cancelación, se verifica si la cola está vacía mediante:

```
qd_is_empty(const QueueDynamic_t* q)
```

Esto evita accesos a memoria inválida cuando no existen trabajos en la cola.

Manejo de Punteros NULL

Los punteros `head` y `tail` se inicializan en `NULL` mediante `qd_init`:

```
q->head = NULL;  
q->tail = NULL;
```

Esto permite identificar fácilmente cuando la cola está vacía y simplifica las condiciones al insertar o eliminar nodos. Además, cuando se elimina el último elemento en `qd_dequeue`, se actualiza `tail` para evitar que apunte a memoria liberada.

Inserción de Trabajos Urgentes

Se implementó una mejora en `qd_enqueue` para manejar prioridad urgente. Si el trabajo tiene prioridad `URGENTE`, el nodo se inserta al inicio de la lista en lugar del final, permitiendo que sea procesado antes que los demás trabajos en espera.

Liberación Segura de Memoria

Debido al uso de memoria dinámica, los nodos se liberan en dos situaciones:

1. Cuando un trabajo se elimina de la cola (`qd_dequeue`).
2. Cuando el programa finaliza (`qd_destroy`).

Esto evita fugas de memoria y asegura una correcta gestión de los recursos del sistema.

Evidencia de Simulación

Para demostrar el funcionamiento del sistema se ejecutó la simulación de impresión completa. Durante la ejecución, el programa muestra en consola el progreso de cada documento página por página, permitiendo observar cómo se reduce el valor de `restante_pgs` hasta que el documento termina de imprimirse.

Ejemplo de salida en consola:

```
--- INICIANDO SIMULACION ---  
  
Imprimiendo documento: reporte.pdf  
Pagina 1 de 5  
Pagina 2 de 5  
Pagina 3 de 5  
Pagina 4 de 5  
Pagina 5 de 5  
Documento reporte.pdf COMPLETADO  
  
--- COLA TERMINADA ---
```

Se tomaron capturas de pantalla durante la ejecución para evidenciar los siguientes aspectos del sistema: inserción de trabajos en la cola, visualización de trabajos pendientes, simulación del progreso de impresión y finalización de los documentos.

Conclusiones

El desarrollo de esta práctica permitió comprender el funcionamiento de las estructuras de datos tipo cola y su aplicación en sistemas reales como la gestión de impresiones.

La implementación inicial con una cola estática permitió comprender los conceptos básicos de almacenamiento y manipulación de datos mediante arreglos. Sin embargo, se observó que esta estructura presenta limitaciones cuando el número de elementos supera el tamaño definido.

La implementación dinámica mediante listas enlazadas permitió construir una estructura más flexible y eficiente, capaz de crecer según las necesidades del sistema. La simulación del proceso de impresión ayudó a visualizar cómo los trabajos avanzan dentro de la cola y cómo el sistema administra el progreso de cada documento.

Las mejoras implementadas, como la prioridad urgente y la cancelación de trabajos, permitieron extender la funcionalidad del sistema más allá de una cola básica.

En general, esta práctica reforzó conceptos importantes de programación en C: el manejo de memoria dinámica, el diseño modular mediante funciones, y el uso adecuado de estructuras y enumeraciones para representar información compleja.

Referencias Bibliográficas

- **Cprogramming.com**. (s.f.). *Pointers in C: Tutorial and Examples*. Recuperado de <https://www.cprogramming.com/tutorial/c/lesson6.html>
- **Gallegos Mariscal, J. C.** (2024). *Práctica 01: Cola de impresión en lenguaje C*. Facultad de Ingeniería, Arquitectura y Diseño, Universidad Autónoma de Baja California.
- **GeeksforGeeks**. (2024, 15 de febrero). *Dynamic Memory Allocation in C using malloc(), calloc(), free() and realloc()*. <https://www.geeksforgeeks.org/dynamic-memory-allocation-in-c-using-malloc-calloc-free-and-realloc/>

- **Kernighan, B. W., & Ritchie, D. M.** (1988). *The C Programming Language* (2nd ed.). Prentice Hall.
- **Sedgewick, R.** (2001). *Algorithms in C, Parts 1-4: Fundamentals, Data Structures, Sorting, Searching* (3rd ed.). Addison-Wesley Professional.
- **TutorialsPoint.** (s.f.). *Data Structures - Queue*. Recuperado de https://www.tutorialspoint.com/data_structures_algorithms/queue_algorithm.htm